QuillAudits

AUDIT REPORT

April 2026

For

PREDIFI

Table of Content

Executive Summary 04

Number of Security Issues per Severity 06

Summary of Issues 07

Checked Vulnerabilities 09

Techniques and Methods 11

Types of Severity 13

Types of Issues 14

Severity Matrix 15

Critical Severity Issues 16

1. Cross-Chain Bridging Reduces totalAssets Without Adjusting Share Supply 16

High Severity Issues 18

2. Stale reward accounting enables share price manipulation across adapters 18

3. Missing Reward Claim Mechanism Causes Permanent Loss of Yield Rewards 19

Medium Severity Issues 20

4. Cross-Contract Role Authorization Missing Protocol-Wide DoS on Settlement and Resolution 20

5. Core Yield and Bridge Functions Permanently Uncallable at Deployment 21

6. Precision Griefing via Low-Decimal Reward Token 22

7. Rival protocol can silently grief LPVault depositors at launch and collapse user trust 24

8. Incorrect Global Principal Accounting Across Multiple Yield Adapters 25

9. LP Vault Withdrawal Logic Enables Race Conditions on Idle Liquidity 27

10. Incorrect Oracle Staleness Check Uses `block.timestamp` Instead of Market Reference Time 28

Low Severity Issues 29

11. Reward Token Misconfiguration Leads to Accounting Corruption and Race Conditions 29

12. MarketRegistry Allows Premature and Arbitrary Market Resolution 31

Informational Issues 32

13. Missing Input Validation in Constructor 32

14. Use of Floating Pragma 33

15. Unbounded feeBps Allows Excessive or Invalid Fee Configuration 34

16. Inconsistent Resolution State Tracking Between Oracle Module and MarketRegistry 35

Functional Tests 36

Automated Tests 43

Closing Summary & Disclaimer 44

Executive Summary

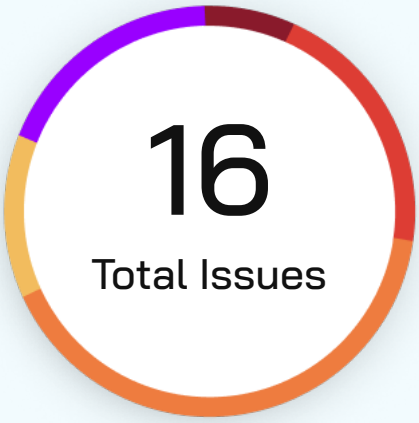
Project Name	Predifi
Protocol Type	Decentralized prediction market protocol
Project URL	https://predifi.com/
Overview	Predifi is a decentralized prediction market protocol that allows users to trade on the outcomes of real-world events and price movements. The protocol is built around a central MarketRegistry that stores and manages both binary and categorical prediction markets, with trading activity settled on-chain through MatchSettlement using EIP-712 signed batch settlements. Market resolution is handled either automatically via the OracleModule which reads price feeds from the Stork aggregator, or manually through AdminOracle for events that have no price feed. All privileged operations across the system are governed by PredifiAdmin, a time-locked multisig-oriented admin contract that enforces a 24-hour upgrade delay and manages role assignments across all governed contracts. The liquidity layer of the protocol operates through two parallel vault systems. PredifiPool is a simple custodial vault where user funds are deposited via per-user UserRouter contracts and managed entirely by an off-chain ledger backend that controls all withdrawals and yield deployment decisions. LPVault is a more sophisticated ERC4626-compliant vault designed for liquidity providers, featuring a Synthetix-style dual reward distribution system, cross-chain USDC bridging via Circle CCTP, and a pluggable yield adapter architecture that currently supports Aave v4 and any ERC4626-compliant vault. Both vaults delegate yield operations to isolated adapter contracts, keeping protocol funds separated from external protocol interactions and limiting blast radius in the event of an adapter-level exploit.
Audit Scope	The scope of this Audit was to analyze the Predifi Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Predifi-com/smart-contracts/tree/main
Branch	main

Contracts in Scope	LPVault.sol PredifiPool.sol ERC4626Adapter.sol AaveV4Adapter.sol MatchSettlement.sol MarketRegistry.sol OracleModule.sol PredifiAdmin.sol AdminOracle.sol RouterFactory.sol UserRouter.sol
Commit Hash	72f5d4cbc9d6d0e50d6a1c407d34b05c34003ab7
Language	Solidity
Blockchain	ethereum , optimism
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	18th march 2026 - 31th march 2026
Updated Code Received	5th April 2026
Review 2	10th April 2026 - 14th April 2026
Fixed In	a3d0fef126f99112565486b14d5d794a8ed747fe

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	1 (6.25%)
High	2 (12.5%)
Medium	6 (43.75%)
Low	2 (12.5%)
Informational	4 (25.0%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Partially Resolved	0	0	0	0	0
Resolved	**1**	**2**	**7**	**2**	**4**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Cross-Chain Bridging Reduces totalAssets Without Adjusting Share Supply	Critical	Resolved
2	Stale reward accounting enables share price manipulation across adapters	High	Resolved
3	Missing Reward Claim Mechanism Causes Permanent Loss of Yield Rewards	High	Resolved
4	Cross-Contract Role Authorization Missing Protocol-Wide DoS on Settlement and Resolution	Medium	Resolved
5	Core Yield and Bridge Functions Permanently Uncallable at Deployment	Medium	Resolved
6	Precision Griefing via Low-Decimal Reward Token	Medium	Resolved
7	Rival protocol can silently grief LPVault depositors at launch and collapse user trust	Medium	Resolved
8	Incorrect Global Principal Accounting Across Multiple Yield Adapters	Medium	Resolved
9	LP Vault Withdrawal Logic Enables Race Conditions on Idle Liquidity	Medium	Resolved

Issue No.	Issue Title	Severity	Status
10	Incorrect Oracle Staleness Check Uses block.timestamp Instead of Market Reference Time	Low	Resolved
11	Reward Token Misconfiguration Leads to Accounting Corruption and Race Conditions	Low	Resolved
12	MarketRegistry Allows Premature and Arbitrary Market Resolution	Informational	Resolved
13	Missing Input Validation in Constructor	Informational	Resolved
14	Use of Floating Pragma	Informational	Resolved
15	Unbounded feeBps Allows Excessive or Invalid Fee Configuration	Informational	Resolved
16	Inconsistent Resolution State Tracking Between Oracle Module and MarketRegistry	Informational	Resolved

Checked Vulnerabilities

- Access Management
- Arbitrary write to storage
- Centralization of control
- Ether theft
- Improper or missing events
- Logical issues and flaws
- Arithmetic Computations Correctness
- Race conditions/front running
- SWC Registry
- Re-entrancy
- Timestamp Dependence
- Gas Limit and Loops
- Exception Disorder
- Gasless Send
- Use of tx.origin
- Malicious libraries
- Compiler version not fixed
- Address hardcoded
- Divide before multiply
- Integer overflow/underflow
- ERC's conformance
- Dangerous strict equalities
- Tautology or contradiction
- Return values of low-level calls

- Missing Zero Address Validation
- Private modifier
- Revert/require functions
- Multiple Sends
- Using suicide
- Using delegatecall
- Upgradeable safety
- Using throw
- Using inline assembly
- Style guide violation
- Unsafe type inference
- Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Cross-Chain Bridging Reduces totalAssets Without Adjusting Share Supply

Resolved

Path

LPVault.sol

Function Name

`bridgeToVenue`

Description

The vault allows a privileged role (BRIDGE_MANAGER_ROLE) to bridge funds to another chain using bridgeToVenue, transferring assets out of the vault without adjusting the share supply.

```
function bridgeToVenue(
                uint32  destinationDomain,
                address recipient,
                uint256 amount,
                bytes calldata bridgeData
            ) external onlyRole(BRIDGE_MANAGER_ROLE) nonReentrant returns (bytes32 messageId) {
                if (amount    == 0)             revert ZeroAmount();
                if (recipient == address(0))    revert ZeroAddress();
                if (!authorizedRecipients[destinationDomain][recipient])
                    revert UnauthorizedRecipient(destinationDomain, recipient);

                address adapter = bridgeAdapters[destinationDomain];
                if (adapter == address(0))      revert NoBridgeAdapter(destinationDomain);

                uint256 idle = IERC20(asset()).balanceOf(address(this));
                if (amount > idle)              revert InsufficientIdle(amount, idle);

                IERC20(asset()).forceApprove(adapter, amount);
                messageId = IBridgeAdapter(adapter).bridgeUSDC(
                    amount,
                    destinationDomain,
                    recipient,
                    bridgeData
                );

                emit LiquidityBridged(destinationDomain, recipient, amount, messageId);
            }
```

Since totalAssets() depends on on-chain balances:

totalAssets = idle balance + adapter deposits

bridging funds out reduces totalAssets() immediately, while:

- No shares are burned
- No accounting adjustment is made

Impact - High

Share Price Dilution

Example:
1. Vault: 10,000 USDC, 10,000 shares → 1:1
2. 5,000 USDC bridged out
3. totalAssets = 5,000, shares = 10,000
4. Share price = 0.5 USDC/share

Result:
• All users suffer a 50% loss on paper

Likelihood

High

POC

1. Vault has 10000 USDC and 10000 shares (1:1)
2. BRIDGE_MANAGER bridges 5000 USDC to another chain
3. totalAssets() drops to 5000, share price drops to 0.5 USDC/share
4. Depositor who had 1000 shares worth 1000 USDC can now only redeem for 500 USDC
5. If bridge fails or funds are lost, the loss is permanent

Recommendation

Burn or Lock Shares Corresponding to Bridged Funds
Alternative approach:
• Treat bridging as a withdrawal
• Burn shares equivalent to bridged assets

This prevents dilution of remaining users.

High Severity Issues

Stale reward accounting enables share price manipulation across adapters

Resolved

Path

LPVault.sol

Function

`totalAssets()`

Description

totalAssets() sums idle balance and all adapter positions via deposited(). For Aave this works correctly because getUserSuppliedAssets() reflects real-time accrual every block. But for adapters where rewards are updated periodically or manually, deposited() returns a stale value — it does not include rewards accrued since the last update. This means totalAssets() is temporarily understated, which directly misprices the LP share.

Impact - High

Share price is understated between reward updates — users depositing during this window receive more shares than they should at fair value
Users withdrawing during this window receive fewer assets than they are entitled to
When the reward update lands, the share price jumps — any attacker who deposited just before the update captures that jump at the expense of existing LPs
With 10 adapters potentially accruing at different cadences, the mispricing compounds — a stale adapter with a large position causes the largest distortion

Likelihood

Medium

Recommendation

a small timelock or entry/exit fee on deposits makes this attack economically unviable, though this is a mitigation rather than a fix.

Missing Reward Claim Mechanism Causes Permanent Loss of Yield Rewards

Resolved

Path

ERC4626Adapter.sol

Description

The protocol integrates an LP vault that deposits user funds into an ERC4626-compatible adapter, which in turn deposits into an external yield-generating vault.
However, if the underlying vault distributes additional rewards (e.g., incentive tokens) that require an explicit claim, there is no mechanism in either:

- the LP vault, or
- the adapter

to claim these rewards.

As a result:
- Reward tokens accumulate in the external vault or adapter layer
- No function exists to retrieve or forward them
- Rewards are effectively orphaned

This breaks the expected yield flow:

```
User → LP Vault → Adapter → External Vault (with rewards)
                                        ↓
                                (unclaimed rewards ❌)
```

Impact - High

Incentive rewards (e.g., liquidity mining tokens) are never claimed

- Users lose a portion of expected yield
- Only base yield (e.g., interest) is realized
- Additional reward emissions are excluded
- Reported or expected APY becomes lower

Likelihood

Medium

Recommendation

Add Reward Claim Functionality in Adapter

Medium Severity Issues

Cross-Contract Role Authorization Missing Protocol-Wide DoS on Settlement and Resolution

Resolved

Path

MarketRegistry.sol, MatchSettlement.sol, AdminOracle.sol, OracleModule.sol

Function Name

`recordTrade(), resolveMarket(), resolveMarketCategorical()`

Description

MarketRegistry.initialize() only grants SETTLER_ROLE and RESOLVER_ROLE to the admin EOA. When MatchSettlement calls recordTrade(), OracleModule calls resolveMarket(), and AdminOracle calls resolveMarket() and resolveMarketCategorical(), the msg.sender in each case is the calling contract address not the admin EOA. Since none of these contract addresses were ever granted the required roles on MarketRegistry, every call reverts. The comment in AdminOracle NatSpec acknowledges this ("This contract must be granted RESOLVER_ROLE on MarketRegistry at deploy time") but nothing in code enforces it.

Impact - Medium

All trade settlements are permanently broken until manually fixed. No market can ever be resolved through OracleModule or AdminOracle. The entire on-chain layer of the protocol is non-functional at deployment. Backend ledger may continue recording trades assuming on-chain confirmation succeeds, creating an accounting mismatch between off-chain and on-chain state.

Likelihood

High

POC

1. Deploy MarketRegistry with admin address A
2. Deploy MatchSettlement pointing to MarketRegistry
3. Register a market via admin
4. Call batchSettle() on MatchSettlement with a valid batch and signature
5. _processFill() internally calls registry.recordTrade()
6. recordTrade() checks onlyRole(SETTLER_ROLE) — msg.sender is MatchSettlement address
7. MatchSettlement was never granted SETTLER_ROLE → revert
8. Same flow for AdminOracle.resolve() → registry.resolveMarket() → Unauthorized revert
9. Same flow for OracleModule.autoResolve() → registry.resolveMarket() → Unauthorized revert

Recommendation

Update MarketRegistry.initialize() to accept the dependent contract addresses and grant them roles at construction time

Core Yield and Bridge Functions Permanently Uncallable at Deployment

Resolved

Path

LPVault.sol

Function Name

`initialize(), deployToYield(), withdrawFromYield(), notifyRewardAmount(), bridgeToVenue(), receiveFromVenue()`

Description

LPVault.initialize() only grants DEFAULT_ADMIN_ROLE and PAUSER_ROLE to the admin address. YIELD_MANAGER_ROLE and BRIDGE_MANAGER_ROLE are defined in the contract but are never assigned to anyone at initialization time. Any call to deployToYield(), withdrawFromYield(), notifyRewardAmount() which require YIELD_MANAGER_ROLE, and bridgeToVenue(), receiveFromVenue() which require BRIDGE_MANAGER_ROLE, will revert with an AccessControl error immediately after deployment. Compare this to PredifiPool.initialize() in the same codebase which correctly grants all four roles to admin at initialization including LEDGER_ROLE and YIELD_MANAGER_ROLE, making the omission in LPVault clearly inconsistent.

LPVault.initialize()grants:grantRole(DEFAULT_ADMIN_ROLE,admin) grantRole(PAUSER_ROLE, admin)

Missing grants: YIELD_MANAGER_ROLE needed by deployToYield, withdrawFromYield, notifyRewardAmount BRIDGE_MANAGER_ROLE needed by bridgeToVenue, receiveFromVenue

Impact - Medium

All yield deployment and withdrawal operations are broken at launch. Reward distribution via notifyRewardAmount cannot be triggered. Cross-chain bridging is completely non-functional. The vault accepts user deposits but the yield management layer that those deposits depend on is entirely locked. No funds are stolen but the protocol's core revenue and liquidity routing mechanisms are dead on arrival.

Likelihood

Medium

POC

- Deploy LPVault with admin address A
- Register a yield adapter via addYieldAdapter — succeeds since it is DEFAULT_ADMIN_ROLE
- Call deployToYield(adapter, amount) — requires YIELD_MANAGER_ROLE
- msg.sender is admin A — admin A was never granted YIELD_MANAGER_ROLE in initialize()
- AccessControl revert
- Same result for withdrawFromYield, notifyRewardAmount, bridgeToVenue, receiveFromVenue

Recommendation

Grant all required operational roles during initialization

Precision Griefing via Low-Decimal Reward Token

Resolved

Path

LPVault.sol

Function Name

`rewardPerToken()`

Description

The contract is vulnerable to precision griefing due to a mismatch between the decimals of the reward token (e.g., 6 decimals) and the staked asset (e.g., 18 decimals).
The rewardPerToken() calculation performs integer division:

```
1
2  function rewardPerToken() public view returns (uint256) {
3      uint256 supply = totalSupply();
4      if (supply == 0) return rewardPerTokenStored;
5      return rewardPerTokenStored
6          + (lastTimeRewardApplicable() - lastUpdateTime)
7          * rewardRate
8          * 1e18
9          / supply;
10 }
11
```

When:
• rewardRate is small (due to 6-decimal token scaling), and
• totalSupply is large (18-decimal precision),
• and updates occur over very short intervals,

the resulting value rounds down to zero.

Example:
• Reward rate ≈ 63 units/sec
• Time interval = 2 seconds
• Total supply = 100,000e18

`(2 * 63 * 1e18) / 100,000e18 = 126 / 100,000 = 0 (rounded down)`

Because of this, calling functions that trigger the updateReward modifier frequently results in:
• rewardPerTokenStored not increasing
• lastUpdateTime being updated regardless

This effectively skips reward accumulation for that time window.

Impact - Medium

An attacker can repeatedly trigger updateReward (e.g., via small deposits or withdrawals) at short intervals such that:
- Each update adds zero rewards due to rounding
- lastUpdateTime keeps advancing
- Legitimate reward accrual periods are continuously reset

Over time:
- A significant portion (or all) of the reward emissions becomes unaccounted for
- These rewards are effectively lost / unclaimable
- Honest stakers receive less rewards than intended

This results in:
- Silent value leakage from the reward pool
- Broken reward distribution fairness
- Protocol-level accounting inconsistency

Likelihood - Medium

- The attack is cheap and permissionless (only requires repeated small interactions)
- No special privileges are required
- However, it depends on:
  - Low reward precision (e.g., 6 decimals)
  - Large total supply
  - Ability to frequently call state-changing functions

Given typical DeFi usage patterns, this is realistic but situational, hence medium likelihood.

Recommendation

Increase Precision in Reward Accounting Use a higher precision scaling factor for tokens with low decimals

Rival protocol can silently grief LPVault depositors at launch and collapse user trust

Resolved

Path

LPVault.sol

Description

LPVault inherits ERC4626Upgradeable and relies on OpenZeppelin's virtual shares mechanism to mitigate the classic ERC4626 inflation attack. However, _decimalsOffset() is never overridden, so it returns the default value of 0, meaning only a single virtual share is added to the denominator:

```
function _decimalsOffset() internal view virtual returns (uint8) {
    return 0; // inherited default, never overridden in LPVault
}
```

This is insufficient for low-decimal assets such as USDC (6 decimals). With only 1 virtual share protecting the denominator, an attacker can manipulate the share price by making a small initial deposit followed by a large direct token donation to the vault — bypassing the `deposit()` function entirely — causing subsequent victim deposits to mint zero shares and losing their funds with no recourse.

Impact - Medium

A victim depositor receives zero shares for a non-trivial deposit, resulting in a complete loss of deposited funds. This is a griefing attack rather than a profitable exploit — the attacker absorbs a small guaranteed loss to inflict a disproportionately larger loss on victims. Since `LPVault` is the core liquidity layer of the protocol, a successful grief at launch permanently damages depositor trust and LP participation:

Likelihood

Medium

Recommendation

Mint dead shares to address(0xdead) at initialisation:

Incorrect Global Principal Accounting Across Multiple Yield Adapters

Resolved

Path

LPVault.sol

Function Name

`totalDeployedToYield`

Description

The contract maintains a single global variable, totalDeployedToYield, intended to track the total principal deployed across all yield adapters. However, this value is updated without regard to which adapter funds are withdrawn from.

When withdrawing from a specific adapter, the contract reduces totalDeployedToYield by the full withdrawn amount (toRecall), even if part of that amount represents yield rather than principal. Since yield accrues unevenly across adapters, this creates a mismatch between the global accounting variable and the actual distribution of funds.

As a result, withdrawing yield-heavy funds from one adapter can incorrectly reduce the accounted principal of other adapters.

Impact - Medium

Breaks a critical accounting invariant: totalDeployedToYield no longer reflects actual principal deployed.
Causes cross-adapter contamination, where yield from one adapter effectively "consumes" principal attributed to another.
Leads to:

- Incorrect internal state
- Misleading risk/accounting assumptions
- Potential downstream logic errors if this variable is relied upon for limits or safety checks

Notably, this does not directly block withdrawals, but it corrupts system integrity.

Likelihood

Medium

POC

Initial state:
totalDeployedToYield = 0
Deploy to Aave: 1000
→ totalDeployedToYield = 1000
Deploy to Compound: 1000
→ totalDeployedToYield = 2000
Yield accrues:
- Aave = 2000 (1000 principal + 1000 yield)
- Compound = 1000 (no yield)
- totalDeployedToYield = 2000 (principal only)

Withdraw from Aave: 2000

Contract logic:
toRecall = 2000
• toRecall <= totalDeployedToYield → true
• totalDeployedToYield -= 2000 → 0

Problem:
• Compound still holds 1000 principal
• But totalDeployedToYield = 0

→ The system has “lost track” of 1000 principal.

Recommendation

Track principal in a way that cannot break the invariant

LP Vault Withdrawal Logic Enables Race Conditions on Idle Liquidity

Resolved

Path

LPVault.sol

Function Name

`withdraw`

Description

The LP vault restricts withdrawals to only the idle balance currently held in the contract, without attempting to pull funds from yield strategies or adapters.

```
function withdraw(
            uint256 assets,
            address receiver,
            address owner
        ) public override nonReentrant whenNotPaused returns (uint256 shares) {
            if (assets   == 0)            revert ZeroAmount();
            if (receiver == address(0))   revert ZeroAddress();

            uint256 idle = IERC20(asset()).balanceOf(address(this));
            if (assets > idle) revert InsufficientIdle(assets, idle);

            return super.withdraw(assets, receiver, owner);
        }
```

This means:

- Users can only withdraw up to the currently available idle liquidity
- Funds deployed in yield adapters are not considered for immediate withdrawal

Impact - Medium

Since withdrawals are limited to idle funds:

- Users compete to withdraw from the same limited pool of liquidity
- The first users to withdraw succeed, while others revert

This creates a clear first-come, first-served race condition, where:

- Early actors can exit successfully
- Later users may be temporarily locked out, even if the vault is solvent

Likelihood

Medium

- No special permissions required
- Occurs naturally under:
  - High utilization (most funds deployed)
  - Periods of high withdrawal demand

Recommendation

Allow withdrawals to pull funds from yield strategies when idle liquidity is insufficient

Incorrect Oracle Staleness Check Uses `block.timestamp` Instead of Market Reference Time

Resolved

Path

OracleModule.sol

Function Name

`_getHistoricalTwap`

Description

The oracle module calculates the freshness (“age”) of historical price data using block.timestamp, rather than the relevant historical reference point (endsAt), which represents the market close time.

```solidity
uint256 age = block.timestamp > ts ? block.timestamp - ts : ts -
block.timestamp;
if (age <= config.maxAgeSec && price > 0) {
```

However, `_getHistoricalTwap()` is explicitly designed to fetch prices around a past timestamp (`endsAt`), not the current block time.

Timestamps used:

```solidity
endsAt - config.twapWindowSec,
endsAt,
endsAt + config.twapWindowSec
```

The staleness check should validate: How close the oracle timestamp `ts` is to the intended historical point (`endsAt`)

Instead, it checks: How close `ts` is to current time (`block.timestamp`)

Impact - High

Any delayed execution will naturally trigger this issue.

Valid Historical Prices Incorrectly Rejected

- If `endsAt` is sufficiently in the past:
- Even perfectly valid historical prices (correct for that time)
- Will appear “stale” relative to `block.timestamp`

Result:

- `validCount` may drop to zero
- Market settlement or logic fails

Likelihood

Medium

Recommendation

Use `endsAt` for Staleness Validation

Low Severity Issues

Reward Token Misconfiguration Leads to Accounting Corruption and Race Conditions

Resolved

Path

LPVault.sol

Function Name

`setRewardsDuration()`

Description

The contract does not validate that the rewardToken is different from the staking asset. As a result, the same token can be configured as both:

```
function setRewardsDuration(
    uint256 duration_
) external onlyRole(DEFAULT_ADMIN_ROLE) {
    if (duration_ == 0)                    revert InvalidRewardsDuration();
    if (block.timestamp < periodFinish)    revert RewardPeriodNotFinished(periodFinish);
    rewardsDuration = duration_;
    emit RewardsDurationUpdated(duration_);
}

```

If rewardToken == asset, reward distribution and vault accounting become intertwined.
The vault calculates total assets as:

```
function totalAssets() public view override returns (uint256) {
    uint256 total = IERC20(asset()).balanceOf(address(this));
    uint256 len   = yieldAdapters.length;
    for (uint256 i; i < len; ++i) {
        total += IYieldAdapter(yieldAdapters[i]).deposited(); // @note check it properly
    }
    return total;
}
```

Since rewards are paid in the same token as the underlying asset:
• Reward tokens held by the contract are indistinguishable from user deposits
• totalAssets() incorrectly includes undistributed rewards as part of principal

This breaks the fundamental separation between:
• User deposits (principal)
• Reward emissions

Impact - Medium

This misconfiguration leads to multiple critical accounting issues:

1. Inflated Share Value (Incorrect Pricing)
   - Undistributed rewards increase totalAssets()
   - Share price becomes artificially inflated
   - New depositors receive fewer shares than they should
   - Existing users benefit unfairly

2. Race Conditions Between Users
Because rewards are part of totalAssets():
   - A user claiming rewards reduces the contract's token balance → decreases totalAssets()
   - A user withdrawing before/after a claim experiences different share value

This creates timing-dependent behavior:
   - Users can front-run reward claims or withdrawals
   - Value can be extracted by acting earlier than others

3. Broken Reward Accounting
   - Rewards are no longer isolated from principal
   - Distribution becomes dependent on vault entry/exit timing instead of staking logic
   - Leads to inconsistent and unfair reward allocation

Likelihood

Low

Recommendation

Enforce Token Separation
Add a strict check in

setRewardToken:
if (token_ == asset()) revert InvalidRewardToken();

MarketRegistry Allows Premature and Arbitrary Market Resolution

Resolved

Path

MarketRegistry.sol

Function Name

`resolveMarket()`

Description

The `resolveMarket()` function in the registry lacks critical validation checks that are enforced in the oracle module. As a result, markets can be resolved:

- Before their close time
- Without using a verifiable price (`closePx`)
- With arbitrary outcomes, including unintended states

In contrast, the oracle module enforces:

- Market must be past close time + dispute buffer
- Outcome must be derived from oracle price (`closePx`) vs strike

Impact - Medium

1. Premature Market Resolution
   - Markets can be resolved while still active
   - Users may still be trading or taking positions

2. Arbitrary Outcome Assignment
   - Resolver can set any outcome manually
   - Outcome is not tied to oracle price (`closePx`)

3. Invalid or Undefined Outcomes
   Although comments suggest valid outcomes are: YES, NO and INVALID

   There is: No enforcement of valid enum states. Potential to resolve to unintended/default values (e.g., `PENDING` if enum is misused)

Likelihood

Low

Recommendation

1. Enforce Time-Based Resolution Constraints
2. Restrict allowed outcomes explicitly:
3. Bind Resolution to Price Data
4. Another option is to lock direct resolution path by making it internal

Informational Issues

Missing Input Validation in Constructor

Resolved

Path

UserRouter
RouterFactory

Function Name

`constructor`

Description

The constructor does not validate critical input parameters (_owner, _vault, _asset). As a result, the contract can be deployed with zero or invalid addresses.

This introduces multiple risks:
• If _owner is set to address(0), all withdraw() calls become permanently inaccessible.
• If _vault is address(0) or incorrect, sweep() will either revert or send approvals to an unintended contract.
• If _asset is invalid, all core functionality (sweep, withdraw) will malfunction or revert.

Because these values are immutable, any misconfiguration at deployment time cannot be corrected.

Recommendation

Add strict sanity checks in the constructor

Use of Floating Pragma

Resolved

Description

The contract uses a floating pragma (^0.8.25), which allows it to be compiled with any future compatible compiler version within the 0.8.x range.

While this provides flexibility, it introduces risk because different compiler versions may include changes in:

- Optimization behavior
- Security checks
- Code generation
- Bug fixes or new bugs

This can result in inconsistent bytecode across deployments, especially when contracts are compiled in different environments or at different times.

Recommendation

Lock the compiler version to a specific

Unbounded feeBps Allows Excessive or Invalid Fee Configuration

Resolved

Path

MarketRegistry

Function Name

`registerMarket`

Description

The feeBps parameter is directly assigned during market registration without any validation:
m.feeBps = p.feeBps;

There is no upper bound enforced on feeBps, allowing values greater than 10,000 basis points (100%), or other unreasonable values.

This can result in markets being configured with excessive or nonsensical fees, potentially breaking economic assumptions or downstream calculations.

Recommendation

Enforce a strict upper bound on feeBps

Inconsistent Resolution State Tracking Between Oracle Module and MarketRegistry

Resolved

Path

OracleModule.sol

Function Name

`autoResolve()`

Description

The oracle module maintains a local mapping `resolvedMarkets` to track whether a market has been resolved:

```solidity
if (resolvedMarkets[marketId]) revert AlreadyResolved();
```

However, this state is not synchronized with the actual source of truth in `MarketRegistry`. If a market is resolved directly via `MarketRegistry.resolveMarket()`, the oracle module remains unaware because `resolvedMarkets[marketId]` is not updated.

As a result:

- `autoResolve()` may proceed under the assumption that the market is unresolved
- It fetches market data and performs all resolution logic
- The transaction ultimately reverts at the registry level due to `MarketAlreadyResolved`

This creates inconsistent state tracking between modules and introduces unnecessary execution flow that is guaranteed to fail.

Recommendation

Use MarketRegistry as the Single Source of Truth

Functional Tests

Some of the tests performed are mentioned below:

MarketRegistry.sol

- registerMarket() correctly creates a new market when a unique marketId is provided, closeTime is in the future, and caller holds MARKET_REGISTRAR_ROLE
- registerMarket() correctly stores all MarketParams fields including question, category, strike, feeBps, resolver, and timing values
- registerMarket() correctly validates CATEGORICAL markets by rejecting fewer than 2 outcomes, more than 16 outcomes, zero outcome IDs, and duplicate outcome IDs
- registerMarket() reverts with MarketAlreadyExists when the same marketId is registered twice
- registerMarket() reverts with InvalidCloseTime when closeTime is in the past or equal to block.timestamp
- registerMarket() reverts with InvalidStartTime when startTime is non-zero and greater than or equal to closeTime
- recordTrade() successfully emits TradeSettled event when market is tradable, buyer and seller are different, and caller holds SETTLER_ROLE
- recordTrade() reverts with MarketNotTradable when market is disabled, resolved, past closeTime, or before startTime
- recordTrade() reverts with SelfTradeNotAllowed when buyer address equals seller address
- resolveMarket() correctly marks a BINARY market as resolved with YES, NO, or INVALID and records resolvedAt timestamp
- resolveMarket() reverts with WrongMarketType when called on a CATEGORICAL market
- resolveMarket() reverts with MarketAlreadyResolved when called on an already resolved market
- resolveMarket() reverts with Unauthorized when caller is neither the market resolver nor holds RESOLVER_ROLE
- resolveMarketCategorical() correctly marks a CATEGORICAL market as resolved when winningOutcomeId matches a registered outcome
- resolveMarketCategorical() reverts with InvalidOutcomeId when winningOutcomeId is not in the market outcomes array

- resolveMarketCategorical() reverts with WrongMarketType when called on a BINARY market
- setMarketDisabled() correctly toggles market disabled flag and emits MarketDisabled event
- isTradable() returns false when market is disabled, resolved, before startTime, or past closeTime
- isTradable() returns true when market exists, is active, not resolved, and within valid time window

MatchSettlement.sol

- batchSettle() correctly processes a valid batch with a valid EIP-712 signature from a BATCH_SIGNER_ROLE holder and emits BatchSettled and FillSettled events
- batchSettle() correctly marks batchId as processed after successful settlement preventing replay
- batchSettle() correctly marks each fill nonce per marketId as processed preventing fill replay
- batchSettle() reverts with DeadlineExpired when block.timestamp exceeds batch.deadline
- batchSettle() reverts with BatchAlreadyProcessed when the same batchId is submitted twice
- batchSettle() reverts with InvalidSignature when signature is from an address that does not hold BATCH_SIGNER_ROLE
- batchSettle() reverts with EmptyBatch when fills array is empty
- batchSettle() reverts with FillAlreadyProcessed when a duplicate fill nonce is included within or across batches
- batchSettle() reverts with InvalidFill when marketId is zero, buyer or seller is zero address, quantity is zero, or price is 10000 or above
- batchSettle() reverts with InvalidFill when the target market is not tradable at settlement time
- pause() correctly halts all batchSettle calls when called by PAUSER_ROLE
- unpause() correctly resumes batchSettle after pause
- batchSettle() reverts when contract is paused

OracleModule.sol

- ✓ autoResolve() correctly resolves a BINARY market as YES when close price is greater than or equal to strike
- ✓ autoResolve() correctly resolves a BINARY market as NO when close price is below strike
- ✓ autoResolve() reverts with TooEarly when block.timestamp is before closeTime plus disputeBuffer
- ✓ autoResolve() reverts with AlreadyResolved when market has already been resolved by this module
- ✓ autoResolve() reverts with UnsupportedSymbol when no SymbolConfig is registered for the market asset symbol
- ✓ autoResolve() correctly uses historical TWAP when Stork atOrBefore is available for the closeTime
- ✓ autoResolve() falls back to current price when historical data is unavailable
- ✓ autoResolve() reverts with StalePrice when latest price age exceeds maxAgeSec
- ✓ setSymbolConfig() correctly stores config and emits SymbolConfigSet when called by CONFIG_ROLE
- ✓ setSymbolConfig() reverts with InvalidSymbolConfig when assetId is zero, maxAgeSec is zero, or targetDecimals exceeds 18
- ✓ setStorkAggregator() correctly updates stork address when called by CONFIG_ROLE
- ✓ previewResolution() returns correct predicted outcome and canResolve flag without modifying state
- ✓ previewResolution() returns alreadyResolved true when market was previously resolved via autoResolve

PredifiAdmin.sol

- ✓ registerContract() correctly adds a governed contract to the registry and emits ContractRegistered
- ✓ registerContract() reverts with AlreadyRegistered when same proxy is registered twice
- ✓ unregisterContract() correctly marks a contract inactive and emits ContractUnregistered
- ✓ execute() correctly calls a function on a registered governed contract and emits Executed
- ✓ execute() reverts with NotGoverned when target is not a registered active contract

- execute() reverts with CallFailed when the underlying call reverts
- batchExecute() correctly processes multiple calls atomically and emits BatchExecuted
- batchExecute() reverts with ArrayLengthMismatch when targets and data arrays have different lengths
- batchExecute() reverts entire batch on first failed call
- queueUpgrade() correctly stores executeAfter timestamp and emits UpgradeQueued
- upgradeContract() correctly upgrades proxy after delay has elapsed and emits ContractUpgraded
- upgradeContract() reverts with UpgradeNotQueued when upgrade was never queued
- upgradeContract() reverts with UpgradeTooEarly when block.timestamp is before executeAfter
- setUpgradeDelay() correctly updates upgradeDelay and emits UpgradeDelayUpdated
- setUpgradeDelay() reverts with InvalidUpgradeDelay when new delay is zero
- grantContractRole() correctly grants a non-admin role on a governed contract when called by OPERATOR_ROLE
- grantContractRole() reverts with DefaultAdminRoleProtected when attempting to grant DEFAULT_ADMIN_ROLE
- revokeContractRole() correctly revokes a role and reverts on DEFAULT_ADMIN_ROLE attempt
- pauseSettlement() correctly triggers pause on registered MatchSettlement contract
- pausePool() correctly triggers pause on registered LPVault or PredifiPool

AdminOracle.sol

- resolve() correctly calls registry.resolveMarket() with YES, NO, or INVALID and emits MarketManuallyResolved
- resolve() reverts when caller does not hold RESOLVER_ROLE on AdminOracle
- resolve() reverts with Unauthorized at the registry level when AdminOracle does not hold RESOLVER_ROLE on MarketRegistry
- resolveCategorical() correctly calls registry.resolveMarketCategorical() with a valid winningOutcomeId and emits MarketManuallyCategoricalResolved
- resolveCategorical() reverts at registry level when winningOutcomeId is not registered for the market
- resolveCategorical() reverts when caller does not hold RESOLVER_ROLE on AdminOracle

LPVault.sol

- deposit() correctly transfers USDC from caller and mints proportional shares to receiver
- deposit() reverts with ZeroAmount when assets is zero
- deposit() reverts when contract is paused
- withdraw() correctly burns shares and transfers idle USDC to receiver
- withdraw() reverts with InsufficientIdle when requested amount exceeds idle balance
- redeem() correctly burns exact shares and returns proportional idle USDC to receiver
- redeem() reverts with InsufficientIdle when previewRedeem exceeds idle balance
- deployToYield() correctly approves and forwards USDC to a registered adapter
- deployToYield() reverts with InvalidYieldAdapter when adapter is not registered
- deployToYield() reverts when YIELD_MANAGER_ROLE is not granted — known initialization bug
- withdrawFromYield() correctly recalls USDC from adapter and updates totalDeployedToYield
- addYieldAdapter() correctly registers an adapter and emits YieldAdapterAdded
- addYieldAdapter() reverts with TooManyAdapters when MAX_YIELD_ADAPTERS limit is reached
- removeYieldAdapter() reverts with AdapterHasBalance when adapter still has deployed funds
- notifyRewardAmount() correctly calculates rewardRate and sets periodFinish when rewardToken is set
- notifyRewardAmount() correctly extends an active reward period by rolling over leftover rewards
- claimRewards() correctly transfers earned rewardToken to caller and zeroes their reward balance
- depositETH() correctly wraps ETH to WETH and mints shares to receiver
- depositETH() reverts with NativeWrapperNotSet when nativeWrapper is not configured
- withdrawETH() correctly burns shares, unwraps WETH, and pushes ETH to receiver
- withdrawETH() reverts with ETHTransferFailed when receiver cannot accept ETH
- setRewardToken() reverts with RewardTokenAlreadySet when called a second time
- setRewardsDuration() reverts with RewardPeriodNotFinished when active reward period has not ended

PredifiPool.sol

- deposit() correctly pulls USDC from caller into pool and emits Deposited event
- deposit() reverts with ZeroAmount when amount is zero
- deposit() reverts when contract is paused
- withdraw() correctly transfers USDC to specified address when called by LEDGER_ROLE
- withdraw() reverts with InsufficientBalance when amount exceeds spot balance
- withdraw() reverts with ZeroAddress when to is zero address
- deployToYield() correctly forwards idle USDC to a registered adapter after asset match validation
- deployToYield() reverts with AdapterNotRegistered when adapter is not in registry
- withdrawFromYield() correctly recalls USDC from adapter back to pool
- registerYieldAdapter() reverts with AdapterAssetMismatch when adapter asset does not match pool asset
- registerYieldAdapter() reverts with TooManyAdapters when MAX_YIELD_ADAPTERS is reached
- removeYieldAdapter() reverts with AdapterHasBalance when adapter still holds funds
- takeYieldSnapshot() correctly stores current TVL and block number and emits TVLSnapshotTaken
- pendingYield() correctly returns difference between current TVL and lastTVLSnapshot
- pendingYield() returns zero when current TVL is less than or equal to lastTVLSnapshot

AaveV4Adapter.sol

- deposit() correctly pulls USDC from PredifiPool, approves Spoke, and calls spoke.supply()
- deposit() reverts with ZeroAmount when amount is zero
- deposit() reverts when called by any address other than owner PredifiPool
- withdraw() correctly calls spoke.withdraw() and transfers received USDC back to owner
- withdraw() correctly exits full position when type(uint256).max is passed
- deposited() correctly returns current balance via spoke.getUserSuppliedAssets()

ERC4626Adapter.sol

- deposit() correctly pulls USDC from owner pool, approves vault, and deposits into ERC4626 vault receiving shares
- deposit() reverts with ZeroAmount when amount is zero
- deposit() reverts when called by non-owner
- withdraw() correctly redeems shares from ERC4626 vault and sends assets directly to owner
- withdraw() correctly exits full position when type(uint256).max is passed using deposited() as withdrawAmount
- deposited() correctly converts held shares to asset value via vault.convertToAssets()
- deposited() returns zero when adapter holds no shares

RouterFactory.sol

- createRouter() correctly deploys a deterministic UserRouter for the caller using CREATE2 and emits RouterCreated
- createRouter() reverts with Unauthorized when msg.sender does not equal user parameter
- createRouter() reverts with ZeroAddress when user is zero address
- computeRouterAddress() correctly returns the address that createRouter would deploy for the same user and salt

UserRouter.sol

- sweep() correctly forwards full asset balance of router to PredifiPool vault and emits ForwardedToVault
- sweep() does nothing when router balance is zero
- sweep() can be called by any address — funds always land in vault
- withdraw() correctly transfers specified asset amount to given address when called by owner
- withdraw() reverts with NotOwner when called by any address other than owner

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of Predifi. We performed our audit according to the procedure described above.

The audit identified multiple issues across varying severity levels, including one critical, two high, seven medium, two low, and four informational findings. All identified issues have been documented and shared with the Predifi team.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

April 2026

For

PREDIFI

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com